树形借用模型

Neven Villani
第一作者
格勒诺布尔大学

Johannes Hostert
第一作者
苏黎世联邦理工大学

Derek Dreyer
作者
马克斯-普朗克
软件系统研究所

Ralf Jung
作者
苏黎世联邦理工大学

Chuigda Whitegive
翻译
Doki Doki λ Club!

Claude
校对
Anthropic

⚠ 注意：本文为早期草稿，内容不完且有措误，且排版质量差。

⚠ Note: this is an early draft. It's known to be incomplet and incorrekt, and it has lots of bad fomatting.

译者前言

本文是文章 Tree borrows 的中文翻译，部分字句有所改动。原文以 CC-BY 4.0 协议发布。术语 *(terminology)* 在正文中第一次出现的地方以仿宋体（中文）或 *Italic (English)* 呈现，如果某个术语难以辨认，则总是会以仿宋体呈现。如遇翻译或排版质量问题，请在 https://github.com/club-doki7/naql/issues 向译者报告。

栈式借用模型 *(Stacked Borrows)* 的概念早在 2018 年就已被提出，并于 2020 年作为学术论文发表并应用于 `miri`。而树形借用模型 *(Tree Borrows)* 的思想也在 2025 年得到了形式化，并且也已经在 `miri` 中得到了实际运用。遗憾的是，这些工作在中文 Rust 社区并未得到应有的广泛认知。许多人将 `unsafe` 视作可以为所欲为的免死金牌，在使用 `unsafe` 时忽视别名规则。译者翻译本文的主要动机之一便是纠正这一常见误解。

摘要

Rust 程序设计语言因其基于所有权的类型系统而闻名，这一类型系统为内存安全和无数据竞争提供了强有力的保证。然而，Rust 也提供了非安全 *(unsafe)* 代码这一"逃生舱门"：在 `unsafe` 的领域中并不自动保障安全性，一切全凭程序设计者自觉。这便引出一种矛盾：一方面，编译器希望能尽可能利用类型系统的强力保证——特别是与指针别名相关的保证——以实现强大的过程内优化；另一方面，这些优化易因"行为不良"的非安全代码而失效。因此，要确保优化的正确性，就必须清晰地定义什么样的非安全代码算是"行为不良"。之前的工作——栈式借用模型——为此定义了一系列规则。然而，栈式借用模型排除了几个在实际的 Rust 非安全代码中常见的模式，且并未考虑到 Rust 借用检查器近期引入的一些高级特性。

为解决这些问题，我们提出了树形借用模型。顾名思义，树形借用模型将栈式借用模型的核心替换为树形结构。这就克服了先前所述的限制：我们评估了 30 000 个最常用的 Rust crate，树形借用模型拒绝的测试用例相比栈式借用模型减少了 54%。此外我们还（在 Rocq 中）证明了新模型在保留栈式借用模型的大部分优化的同时，还让一些重要的新优化成为了可能——特别是读-读重排序 *(read-read reordering)*。

1. 简介

类型系统已被证明能以可组合的方式高效地排除整类错误。Rust 正是最近的成功案例，它使用仿射 *(affine)*类型（"基于所有权的"类型）和生存期 *(lifetime,* 区域类型 *region types* 的变体*)* 来静态地确保底层系统应用程序的类型安全、内存安全和无数据竞争，而无需垃圾回收器。

然而，类型系统的好处不只是确保程序不会出错：类型系统还能提高程序的运行速度。事实上，Rust 编译器从一开始就将其类型系统用于优化，而不只是确保安全。特别地，Rust 的类型中编码了强大的别名信息：可变不共享，共享不可变。别名分析是现代优化器的核心支柱，因此 Rust 编译器的开发者希望将无处不在的类型信息用于别名分析也就不足为奇了。考虑这个具体的例子：

```rust
fn write_both(x: &mut i32, y: &mut i32) -> i32 {
    *x = 13;
    *y = 20;
    *x // 返回 13
}
```

例 1

函数 `write_both` 接受两个可变引用作为实参，而根据 Rust 的类型系统规则，这两个引用不可能互为别名。由此可知，该函数必须始终返回 `13`。因此最后一行中从 `*x` 读取数据的操作可被移除，返回值可被硬编码为 `13`，而这些变换不会影响程序的行为。虽说这个例子是人为设计的，但它展示了一类强大的过程内优化：无论内存是在程序中何处分配的，编译器总是可以作这种变换。

1.1. 非安全代码带给别名优化的挑战

不幸的是，Rust 所面临的现实比这复杂得多。Rust 的类型系统所提供的别名保证有时过于强大，使得程序设计者无法实现某些抽象（例如，依赖于共享状态变更的抽象）或是达到预期的性能。因此，程序设计者有时需要使用非安全代码——尤其是原始指针 *(raw pointer)*——来规避类型系统的限制，而原始指针的别名是不受追踪的。Rust 程序设计者会将非安全代码封装在安全的抽象层中以保持类型安全推理的组合性。例如，标准库中的 `Vec` 类型提供了大小可变的数组。`Vec` 类型内部是用非安全代码实现的，但它提供给用户的公共 API 却是安全的。这意味着数百万行代码可以使用 `Vec` 而无需担心类型安全问题，只有实现 `Vec` 的那几千行代码需要额外的审查。

非安全代码的问题在于，用它们可以进行如下操作：

```rust
fn main() {
    let mut x = 42;
    let ptr = &mut x as *mut i32;
    let val = unsafe { write_both(&mut *ptr, &mut *ptr) };
    println!("{val}");
}
```

例 2

以上代码先是创建了指向可变局部变量 `x` 的原始指针 `ptr`，然后用 `unsafe` 代码块，两次将同一个原始指针转型为可变引用，这样就有了两个互为别名的可变引用。因此若不带优化地编译，则这段程序会输出 `20`，但若按照先前所述的方法优化 `write_both`，则程序会输出 `13`。坏！优化不应该改变程序的行为。

Rust 编译器的开发者确是希望支持别名优化的，这就需要某种方法来“排除”上面这样的反例。Rust 程序设计者早已习惯了这一点：使用非安全特性不意味着他们可以为所欲为，编写非安全代码的开发者必须格外注意特殊规则。例如，调用 `get_unchecked` 函数以在不进行边界检查的前提下访问数组时，非安全代码的作者必须确保索引在数组边界之内。不符合这些要求的非安全代码包含未定义行为 *(Undefined Behavior, UB)*，因为由此产生的程序可能会以任意方式出现异常行为。因此，非安全代码的作者有义务确保他们的代码没有未定义行为；而编译器会反过来信任程序设计者，并假定它所编译的代码中没有未定义行为。

要保留例 1 中基于引用的优化，我们需要找到一种方法，使得例 2 中的程序具有未定义行为。但这说来容易做来难。例如，以下完全安全的 Rust 代码片段看似也会导致可变引用别名问题：

```rust
let mut x = 42;
let ref1 = &mut x;
let ref2 = &mut *ref1;
*ref2 = 12;
println!("{ref1}"); // 输出 12
```

例 3

ref1 和 ref2 都指向 x。Rust 若是断然不许可变引用互为别名，又怎能允许这种状况呢？因为这是一种重借用 *(reborrowing)*：ref2 派生自 *(derived from)* ref1，故编译器允许它们同时存在。然而当 ref2 存活时，ref1 便不能使用；而当 ref1 再被使用时，即是 ref2 生存期的终点。若在例 3 的结尾添上一行 `println!("{ref2}")`，就违反了这规则，程序便编译不成了。

因此，摆在我们面前的挑战便是：如何定义"何谓未定义行为"，使得形如例 2 的反例被拒绝，但所有安全 Rust 代码都被接受。事实上，我们还要更进一步，确保足够多"合理"的非安全代码也被接受：必须让程序设计者能以适度的努力编写出正确的非安全代码，非安全 Rust 才真正称得上可用。

1.2. 先前的工作：栈式借用模型

我们不是第一批勇攀高峰的人：栈式借用模型 [12] 有着完全相同的目标。栈式借用模型注意到，在例 3 这样的场景中，当前有效的指针可被记录在一个栈中：x 在栈底，ref1 在它上面，而 ref2 在栈顶。栈模型是对这一事实的建模：引用的所有使用必须是良嵌套的 *(well nested)*。当 ref1 被使用时，ref2 就会失效。不幸的是，简单的栈模型有三大问题。

栈式借用模型的第一个问题在于它会阻止一些基本的优化，例如重排序相邻的读取操作。原因很简单：在例 3 中，若先读取 ref2 再读取 ref1，那么一切相安无事。ref2 是"嵌套于" ref1 中的，所以所有访问都是允许的。然而，若重排序这两个操作，就成了先读取 ref1 再读取 ref2。这在栈式借用模型下是未定义行为，因为这违反了良嵌套规则。读取重排序是一项至关重要的优化，因此这是一个显著的弊端[1]。

栈式借用模型的第二个问题在于它将引用限定在一个静态范围内：引用被创建之后，引用的类型决定了它所能访问的内存范围，无论是引用本身还是由它所派生出的指针，均不能访问这一范围外的内存。而在实践中，这往往并非程序设计者所期望的行为。例如，有 C 语言经验的程序设计者有时会创建指向数组首元素的引用，将其转成原始指针，然后用指针访问整个数组。栈式借用模型会拒绝这种操作，因为从指向数组首元素的引用派生出的原始指针只能访问这一个元素。程序设计者的心智和实际模型之间的这一差异导致现实代码中频繁出现未定义行为 [1, 3, 5, 6, 8]，因此，一个更宽松的替代选项呼之欲出。

第三个问题则相当技术性，它涉及一个与栈式借用模型同期开发的 Rust 借用检查器特性：两阶段借用 *(two-phase borrows)*。这一特性弱化了对某些可变引用的要求：它们不必在创建之后马上就是唯一的；相反，它们在被创建时处于一个保留 *(reserved)* 阶段，这一阶段允许只读别名。只在可变引用第一次被写入前、被激活 *(activate / activation)* 时，唯一性才被强制执行。

原本的栈式借用模型完全不支持两阶段借用。Rust 程序设计者用栈式借用模型来检查代码中的未定义行为，而栈式借用模型的实现只对两阶段借用提供了简陋的支持：在检查含有两阶段借用的代码时，实现基本上是把两阶段借用当作原始指针处理的。这一简陋支持会产生一些非常反直觉的效果，时常令 Rust 程序设计者感到困惑，如下例所示：

[1]除了整个换掉栈式借用模型之外，这一缺陷还可通过为优化所操作的中间表示形式选用不同的模型来克服。但这仍然需要开发一个新模型，而因为该模型是栈式借用模型的细化，其必然受到更多约束——即中间表示模型具有的未定义行为只能比表层模型更少。为此，我们决定完全替换掉栈式借用模型。

```rust
fn write(x: &mut i32) { *x = 10; }

let x = &mut 0;
let y = x as *mut i32; // 从 x 中派生原始指针
write(x);              // 写入 x
unsafe { *y = 15 };    // 使用原始指针
```

例 4

栈式借用模型接受以上程序。然而，若将 `write` 调用替换为其函数体，则栈式借用模型会拒绝该程序[2]。造成这一区别的原因在于 `write(x)` 隐式地展开成了 `write(&twophase x)`，这里 `&twophase` 是一个伪语法，表示受两阶段借用规则约束的隐式可变引用。而栈式借用模型会将两阶段借用当作原始指针处理。这一差异不但会让程序设计者感到困惑，将隐式创建的可变引用视作指针也会限制优化潜力。

1.3. 我们的贡献：树形借用模型

在本文中，我们展示了一种能够克服以上局限的新别名模型，我们称之为**树形借用模型**：

- 要支持两阶段借用，就需要使用树代替栈来记录引用间的关系。栈结构已能确保每个引用都有唯一的父级：创建它的引用 / 局部变量。但与此同时，栈结构还强加了一个额外的约束：每个引用在同一时刻至多只能有一个子级是有效的。而这在处理两阶段借用时就力不从心了：处于保留阶段的可变引用可与多个共享引用和平共处。因而这些引用必须被作为同一个父级引用的独立子级记录，而当可变引用被激活时，其同级引用就不能再使用了。这样一来，使用树形结构也就顺理成章。
- 为支持读取重排序，树上的每个节点都记录一个状态机，该状态机表示由该节点表示的引用的当前权限。当该引用所指向的内存经由该引用或其他引用读写时，权限会相应地改变。这与栈式借用模型截然相反：栈式借用模型中，引用的权限在创建时便已固定，直至失效都不会改变。
- 最后，栈式借用模型的静态引用范围也被改为动态引用范围，引用声称唯一性（或者不变性）的内存区域不再有固定的界限。区域现在由引用的使用方式决定。这很大程度上依赖于树形结构：树形结构可以跟踪多个同级引用，而无需预先决定哪个引用对于内存的哪一部分是唯一的。

为了实证树形借用模型的有效性，我们在中层中间表示形式解释器 *(Mid-level Intermediate Representation Interpreter, Miri)* 中实现了这一模型——Miri 也曾被用于评估栈式借用模型。我们在下载数最多的 30 000 个 Rust 库上开展了实验，确认树形借用模型所拒绝的测试用例比栈式借用模型少 54%。我们也在 Rocq 中形式化建模了树形借用模型，并用 Simuliris [9] 框架构建了一个关系程序逻辑，证明了树形借用模型优化的正确性。栈式借用模型上生效的几乎所有优化都在树形借用模型上生效；唯一的例外是在原始程序的第一次写入之前插入新写入的优化——而这是为了让树形借用模型与现实世界的代码更兼容所作的权衡之一。据我们所知，树形借用模型支持 Rust 编译器已在运用的别名优化（如例 1 所示）；而不同于栈式借用模型的是，在树形借用模型下能够证明重排序相邻的读取操作是健全的。

本文中的其余部分结构如下：我们首先以示例驱动的方式介绍树形借用模型的核心（第 2 节）。接下来，我们用保护器 *(protector)* 扩展树形借用，这将显著增强可以执行的优化（第 3 节）。之后，我们将新模型与其前身——栈式借用模型——进行详细比较，包括上面提到的实证评估（第 4 节）。最后，我们将展示一些已证明正确的树形借用模型下的优化案例（第 5 节），并以对相关和未来工作的讨论收尾（第 6、7 节）。

[2]这看似是栈式借用模型不允许内联，但实际情况更加微妙。行为上的差异源于表层 Rust 被繁饰 *(elaborate)* 为内部中间表示形式的方式不同；而中间表示形式级别的内联对于栈式借用模型而言是健全的。

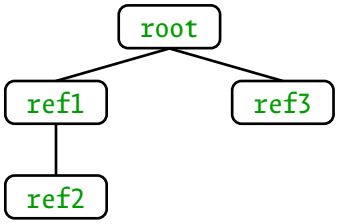
2. 树形借用模型基础

在本节中，我们将解释树形借用模型追踪引用和检测未定义行为背后的基本机制。我们不假设读者事先了解 Rust 或栈式借用模型。

2.1. 记录别名

树形借用模型的核心思想正如其名：引用被组织成一个树形结构。例如，考虑这个例子：

```
let mut root = 42;
let ref1 = &mut root;
let ref2 = &mut *ref1;
let ref3 = &mut root;
```



例 5

例子中有一个局部变量 `root` 和几个或直接或间接地派生自该变量的引用。右边的树状图展示了树形借用模型如何表示这些引用与局部变量之间的关系。每个新创建的引用都与树上的一个新节点关联，并作为一个子节点插入，其父级是用于创建该引用的节点。因此，引用不仅仅是内存中的一个位置；相反，它是由一个位置和一个标识符所构成的二元组来定义的，该标识符确定了其在树中对应的节点。我们将这一标识符称为引用的“标签”。

访问的效应　每当内存访问发生时，树形借用模型会将这次访问“通知”给所有引用。每个引用——也就是树上的每个节点——记录一个状态机，该状态机定义了引用应对该访问作出何种“反应”：访问要么被批准（可能会改变状态机的状态），要么被拒绝——也就是程序包含未定义行为。

状态机的状态转移由两个因素决定：一是访问操作是读操作还是写操作；二是作出“反应”的引用与用于此次访问的引用之间是何关系。特别地，我们区分*局部访问 (local accesses)* 和*外部访问 (foreign accesses)*。在访问标签 t_{acc} 之后、计算标签为 t_{sm} 的节点的状态机转换时，若 t_{acc} “派生自” t_{sm}——即 t_{acc} 是 t_{sm} 自身或其子节点，则称该访问时对 t_{sm} 的局部访问；若 t_{acc} 是 t_{sm} 的父节点或兄弟节点，则称该访问为外部访问。例如，在上面的例 5 中，设 t_{sm} = `ref1`，则对 `ref1` 和 `ref2` 的访问就是局部访问，而对 `ref3` 或 `root` 的访问就是外部访问。总的来说，状态机的字母表被定义为 { 局部读 (↓R)，局部写 (↓W)，外部读 (↑R)，外部写 (↑W) }。

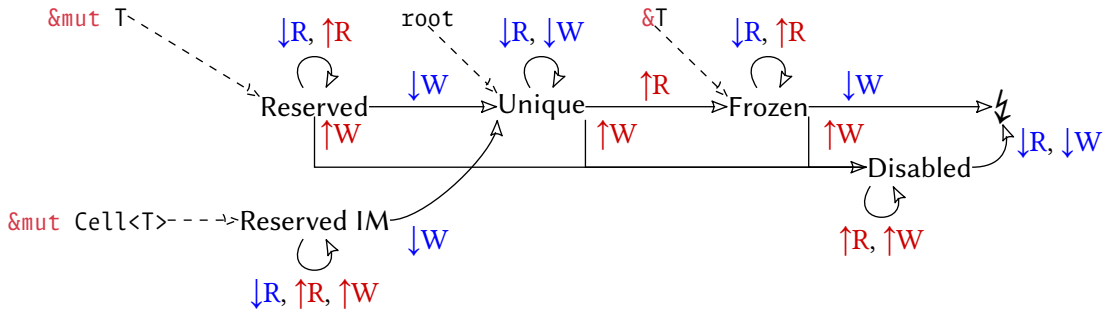


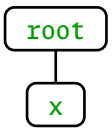
图 1　权限的默认状态机，可变引用入口点由 `&mut T` 标记。抵达状态 ↯ 表明程序包含未定义行为。转移箭头上的标签代表导致该转移的事件：读 (R) 或写 (W)，↑外部或↓局部。

2.2. 可变引用的生存期

现在我们开始逐步定义这个状态机，状态机的完整版本已由图 1 给出。状态机的大部分内容都可通过探讨可变引用来理解。因此，我们从一个简化的可变引用模型开始，随后对其进行扩展，直至该模型能够接受所有安全代码，并允许我们所需的优化。

可变引用的朴素模型　为了从如何处理可变引用中汲取灵感，我们首先将目光投向 Rust 编译器中负责确保引用正确使用的组件——借用检查器（borrow checker）。让我们从一个非常简单的示例开始：

```
let mut root = 42;
let x = &mut root;
*x += 1;
root = 0;
```



例 6

这里，可变引用 `x` 由表达式 `&mut root` 创建，故 `x` 是 `root` 的子级。借用检查器将决定 `x` 的生存期始于 `&mut root`，且无法在 `root = 0` 之后延续——那时，父级引用将重新取得所有权，这必然会导致 `x` 被终止。这一期望的行为可以很容易地用我们的权限和局部/外部访问框架表达：我们使用两个权限 Unique 和 Disabled 表示可变引用的生死。可变引用在一开始被赋予权限 Unique，且只要其权限是 Unique，它就能容许任意的局部访问。而当外部访问发生，也就意味着父级引用收回了指涉物的所有权，故权限应转移至 Disabled，从此刻开始，任何对 `x` 的局部访问都会触发未定义行为。